



Introduction to Databases

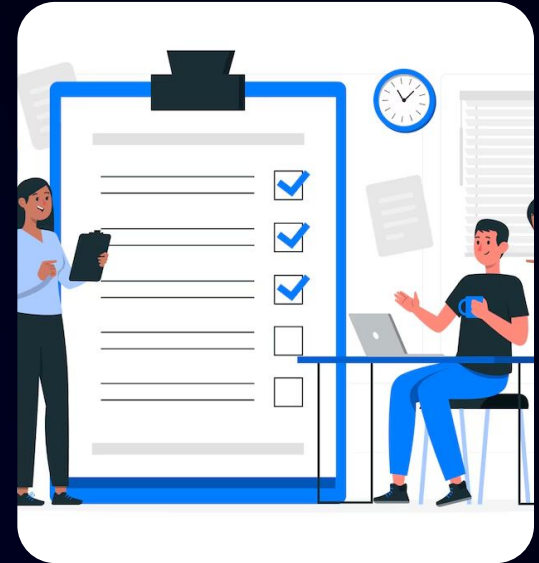
Learning Objectives

- Understand the evolution of databases and their critical role in storing and retrieving data.
- Differentiate between raw data and information and how databases facilitate meaningful insights.
- Explore the concept and architecture of Database Management Systems (DBMS).
- Gain an overview of various database types and their characteristics, including relational and NoSQL models.
- Compare relational and non-relational databases in terms of structure, scalability, and use cases.
- Identify real-world applications where specific database types excel.



Prerequisites

- Basic Understanding of Programming
- Knowledge of Data Structures
- Concepts of File Management and Data Storage
- Logical and Analytical Thinking

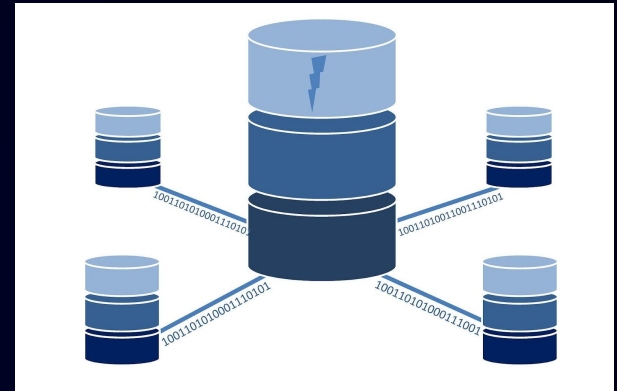




Introduction to Databases

Definition and Overview of Databases

- A database is an organized collection of data that can be easily accessed, managed, and updated.
- Typically controlled by a Database Management System (DBMS), databases store data in structured formats, often using tables composed of rows and columns.
- This organization allows for efficient querying and processing of data, making databases essential for various applications such as banking, education, and e-commerce



Evolution of Databases

Flare files



1960

PROS

Simple Structure

CONS

No simple way to query data

Hierarchical
database



1970

PROS

Better than files
in querying

CONS

Update were tedious

relational
database



1980

PROS

Easier to query
and update

CONS

Not designed for
web 2.0 scale

NoSQL



2005

PROS

Excellent at scale

CONS

No transactions

polyglot
persistence



Today

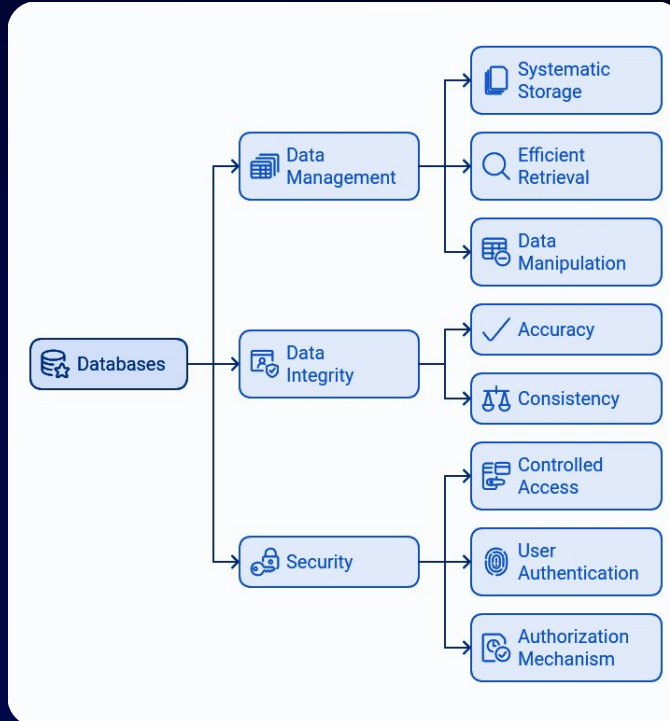
PROS

Levelage
Capabilities of
different Database

CONS

Need to manage different
Types of DB

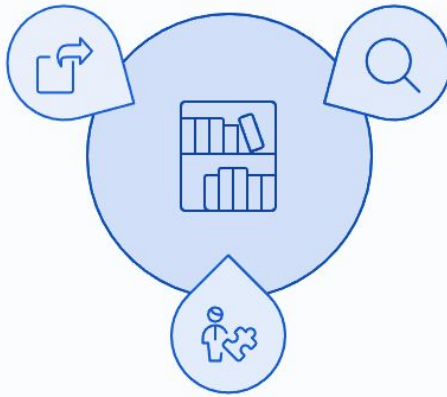
Importance of Databases



Impact on Data Storage and Retrieval

Data Sharing

Facilitates consistent and integral data sharing across applications



Quick Access

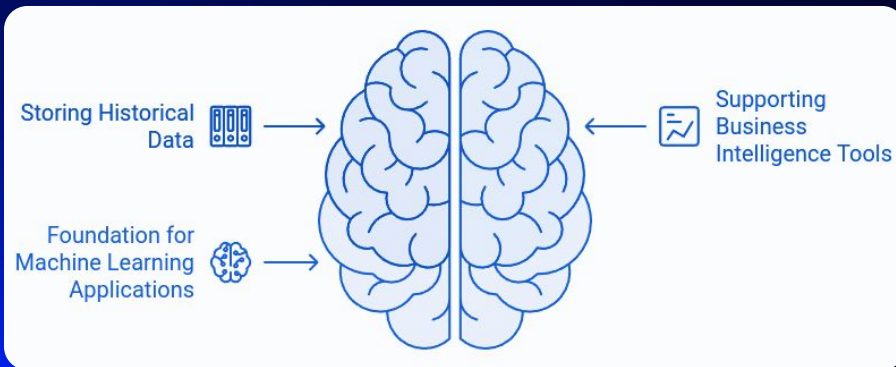
Enables fast retrieval of information through indexing and optimized queries

Complex Queries

Supports intricate queries to extract insights from diverse datasets

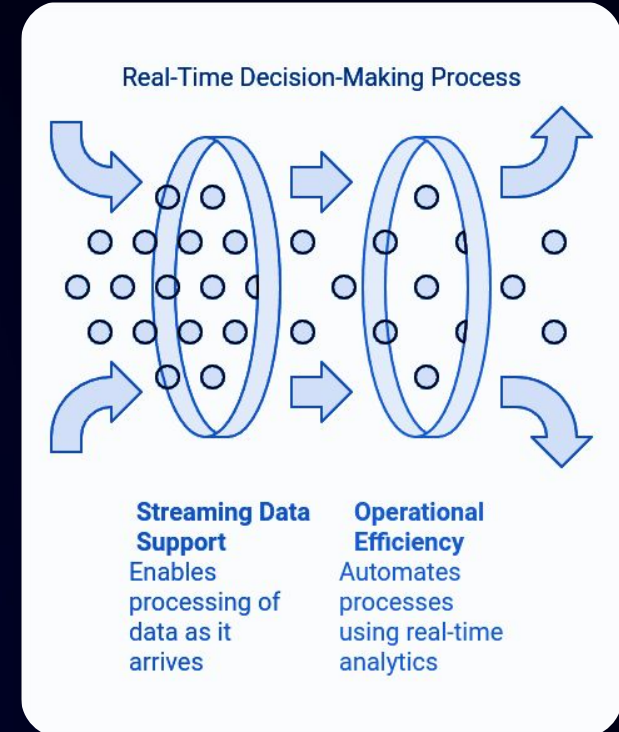
Enabling Advanced Analytics

- Storing historical data that can be analyzed to identify trends and patterns.
- Supporting business intelligence tools that help organizations make informed decisions based on data-driven insights.
- Providing a foundation for machine learning applications where large datasets are essential for training models



Real-Time Decision-Making

- Allowing instant access to current data, which is critical for industries like finance where timely information can impact trading decisions.
- Supporting streaming data technologies that process information as it arrives, facilitating immediate responses to changing conditions.
- Enhancing operational efficiency by automating processes based on real-time analytics



Pop Quiz

Q. What is a database?

A

A method for securing data in files

B

A collection of data organized for easy access, management, and updating

Pop Quiz

Q. What is a database?

A

A method for securing data in files

B

A collection of data organized for easy access, management, and updating



Data, Information, and Databases

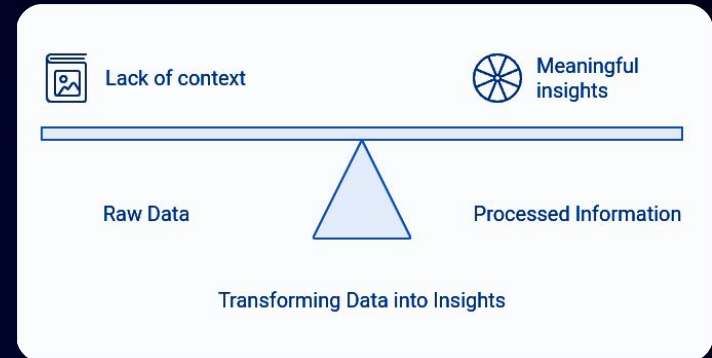
Raw Data vs Processed information

Raw Data:

- Unprocessed facts and figures without context, such as numbers, dates, and strings.
- For example, a list of temperatures recorded over a week.

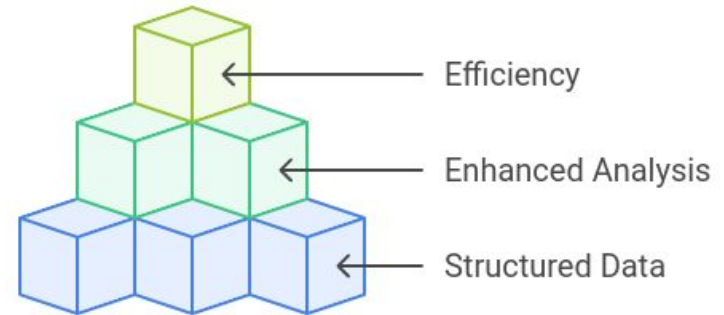
Processed Information:

- Data that has been organized, structured, or presented in a meaningful context.
- For instance, the average temperature calculated from the raw data over the week provides insights into weather trends.

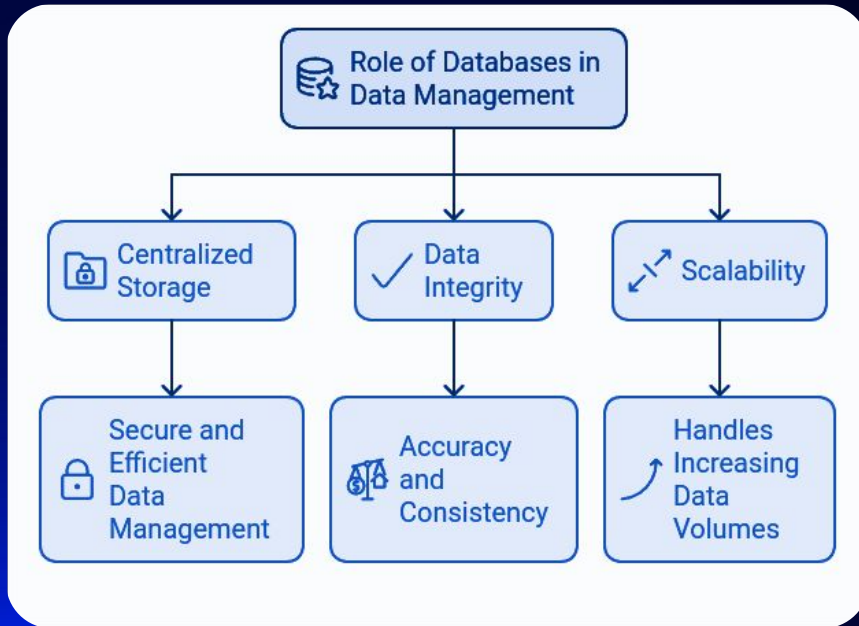


Importance of Data Organization in Decision-Making

- **Structured Data:** Well-organized data facilitates easier access and retrieval, which is essential for effective decision-making.
- **Enhanced Analysis:** Organized data allows for comprehensive analysis, enabling stakeholders to draw insights that inform strategic decisions.
- **Efficiency:** Proper data organization reduces the time needed to find information, leading to quicker responses to business needs.



Role of Databases in Data Management



Examples of Data vs. Information

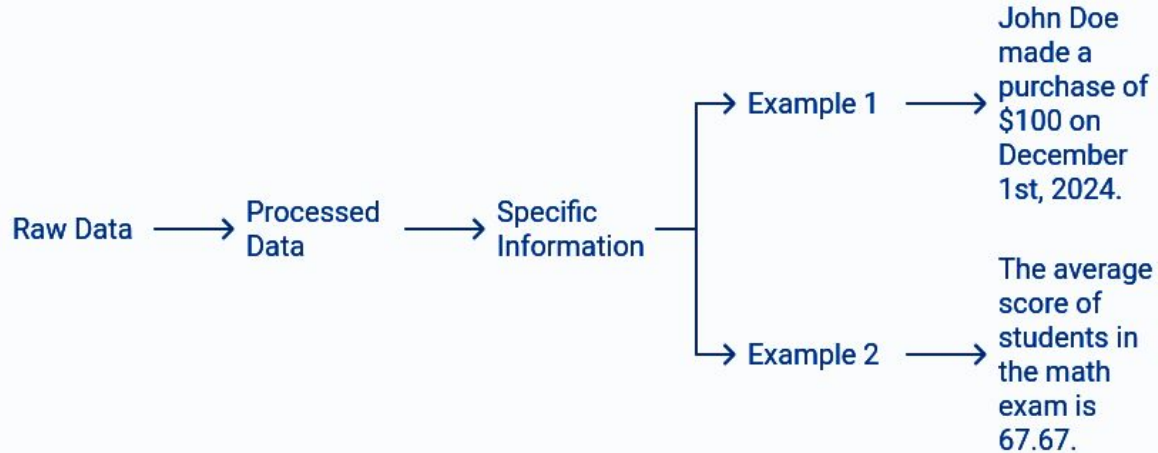
- **Example 1:**

- **Data:** "2024-12-01", "John Doe", "100"
- **Information:** "John Doe made a purchase of \$100 on December 1st, 2024."

- **Example 2:**

- **Data:** "35", "78", "90"
- **Information:** "The average score of students in the math exam is 67.67."

Examples of Data vs. Information



Pop Quiz

Q. Which of the following is an example of unstructured data?

A

A text document or image file

B

A relational database table

Pop Quiz

Q. Which of the following is an example of unstructured data?

A

A text document or image file

B

A relational database table



Database Management Systems (DBMS))

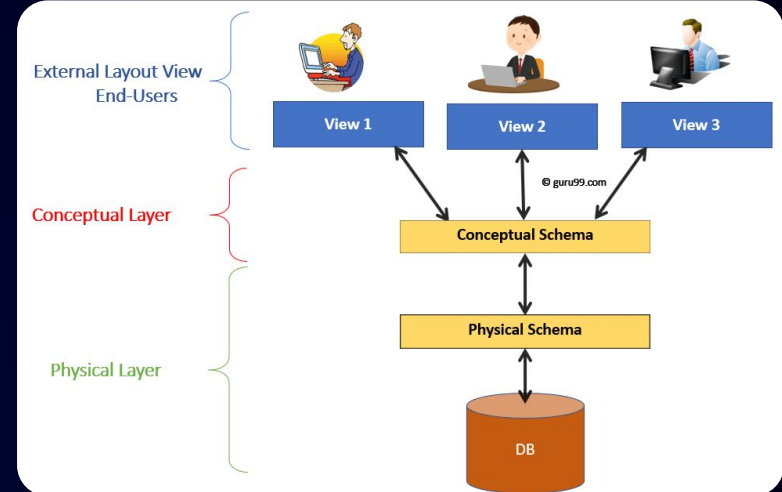
Definition of DBMS

- A Database Management System (DBMS) is a software system designed to create, manage, and manipulate databases.
- It serves as an interface between users and the database, allowing for operations such as data creation, retrieval, updating, and deletion.
- DBMS ensures data security, integrity, and consistency while providing a structured environment for data management



Understanding Architecture of a DBMS

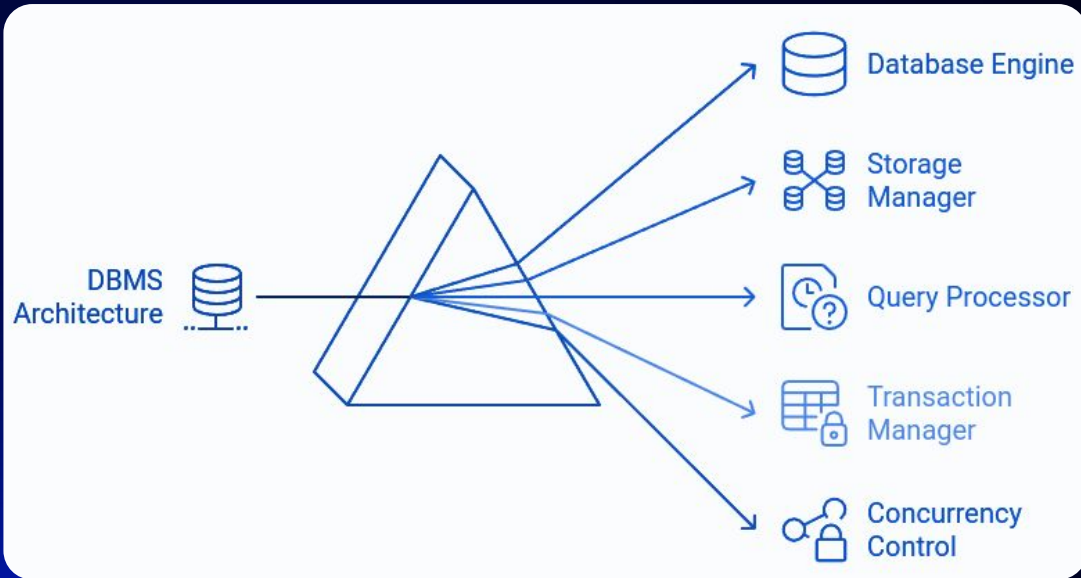
- DBMS architecture refers to the structural design and components that work in harmony to manage and maintain databases effectively.
- One common approach is the client-server architecture, which separates client and server components to facilitate data handling, application logic, and user interactions.



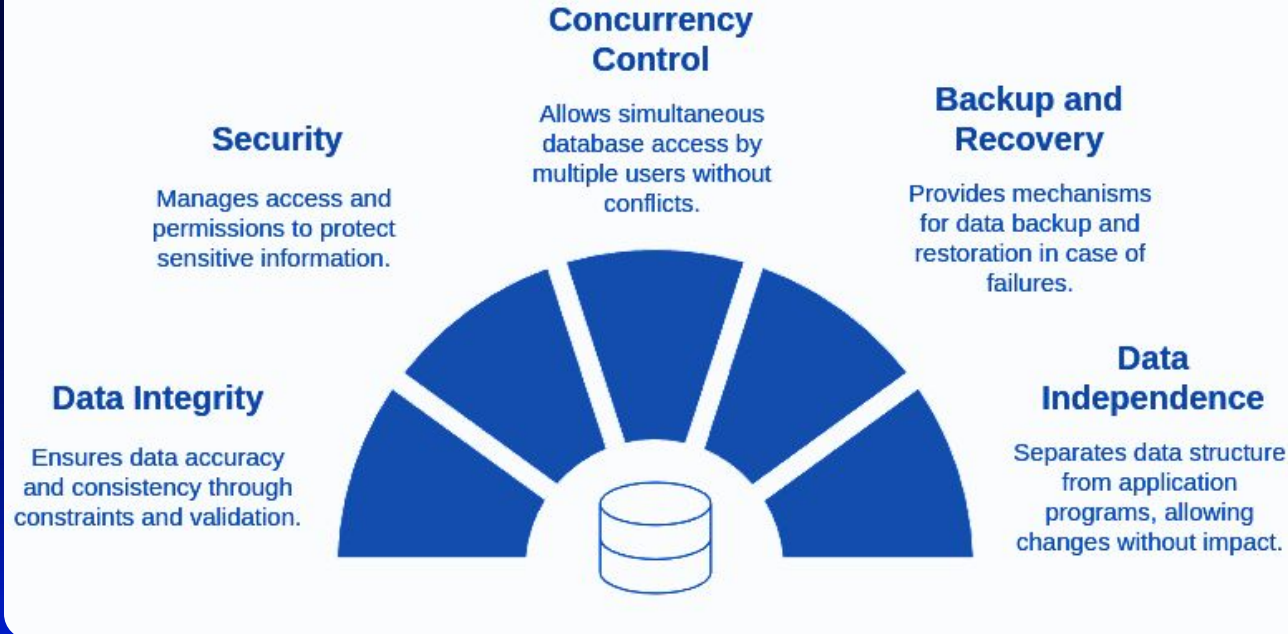
Core Architecture of a DBMS

- Here are the five core components of a DBMS architecture with brief descriptions:
 - **Database Engine:** The core service responsible for managing data storage, retrieval, and processing.
 - **Storage Manager:** Handles the organization, storage, and retrieval of data on physical storage devices.
 - **Query Processor:** Interprets and executes database queries, optimizing performance where possible.
 - **Transaction Manager:** Ensures database transactions are processed reliably and adhere to ACID properties.
 - **Concurrency Control:** Manages simultaneous data access by multiple users to maintain consistency and prevent conflicts.

Core Architecture of a DBMS



Common Features of DBMS



Examples of Popular DBMS

- Here are some widely-used Database Management Systems:
 - **MySQL:** Open-source, fast, and reliable, popular for web applications.
 - **Microsoft SQL Server:** Robust RDBMS by Microsoft, ideal for enterprise use.
 - **Oracle Database:** Scalable RDBMS for complex transactions.
 - **PostgreSQL:** Advanced open-source database with extensibility and standards compliance.
 - **MongoDB:** NoSQL database for unstructured data and high scalability

Examples of Popular DBMS



ORACLE®
DATABASE



Pop Quiz

Q. Which of the following is NOT a function of a DBMS?

A

Data storage and retrieval

B

Designing hardware for databases

Pop Quiz

Q. Which of the following is NOT a function of a DBMS?

A

Data storage and retrieval

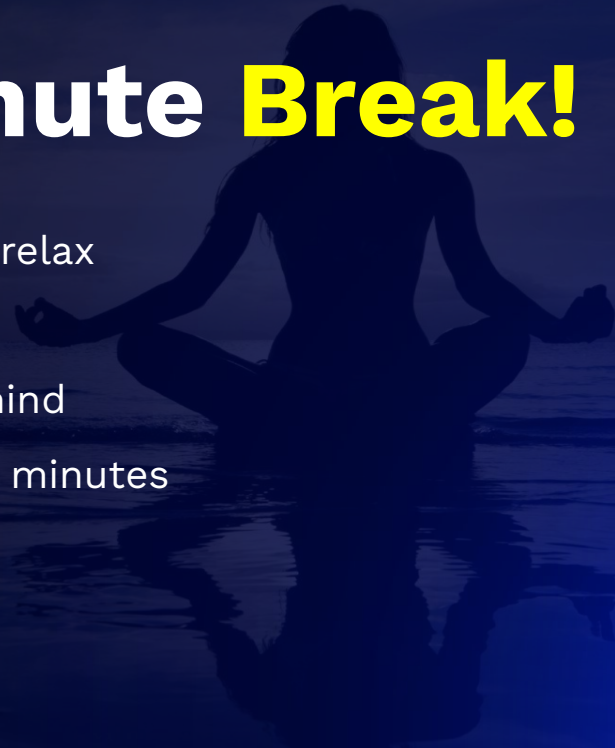
B

Designing hardware for databases



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes

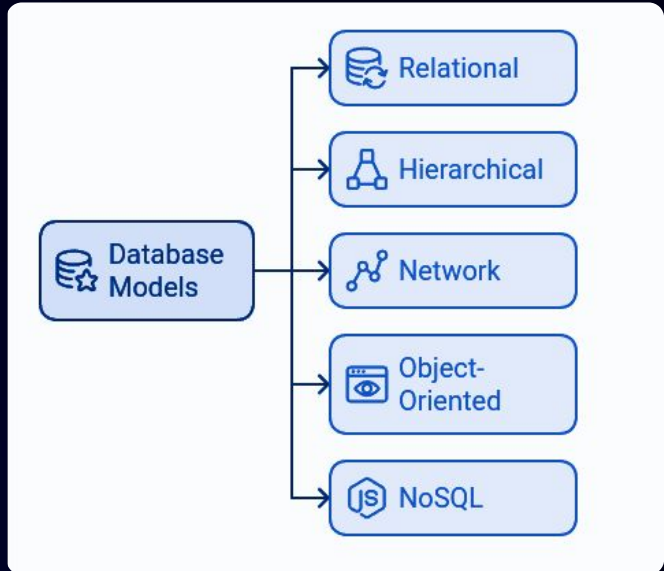




Types of Databases

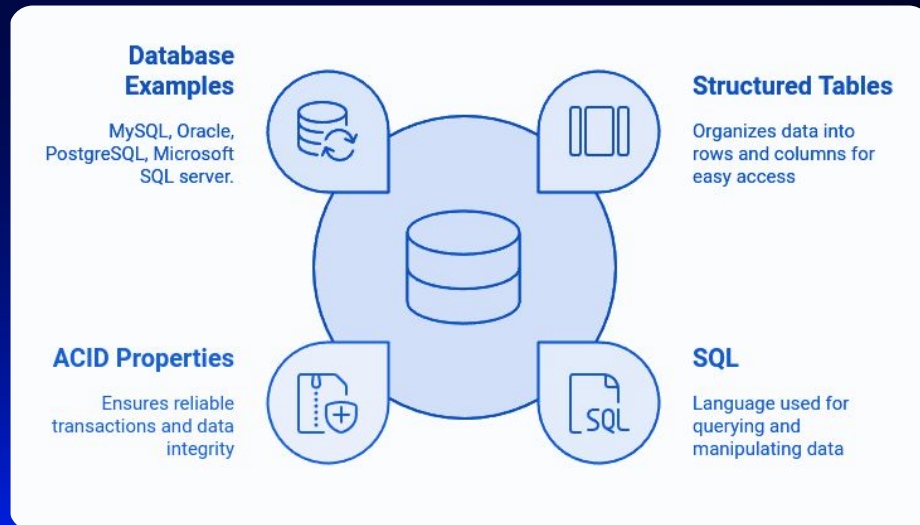
Overview of Database Models

- Databases can be categorized into various models based on their structure and data management approach.
- The primary types include
 - Relational,
 - Hierarchical,
 - Network,
 - Object-Oriented, and
 - NoSQL databases.
- Each model has distinct characteristics that make it suitable for specific applications and use cases.



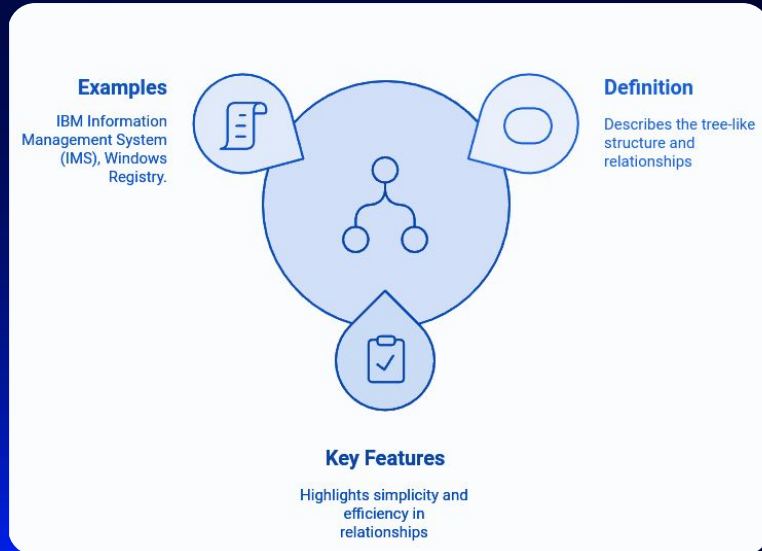
Relational Databases (RDBMS)

- A type of database that organizes data into structured tables (relations) with rows and columns, where relationships between data are defined using keys, ensuring data integrity and enabling complex queries.



Hierarchical Databases

- A database model that organizes data in a tree-like structure, where data is stored in parent-child relationships, and each child record has only one parent, making it ideal for data with a clear hierarchy.



Network Databases

- A database model that organizes data using a graph structure, allowing multiple parent-child relationships, where records are connected through links, enabling more complex relationships than hierarchical databases.

Definition

Describes the structure allowing multiple parent-child relationships.

Flexibility

Highlights the adaptability in representing complex relationships.

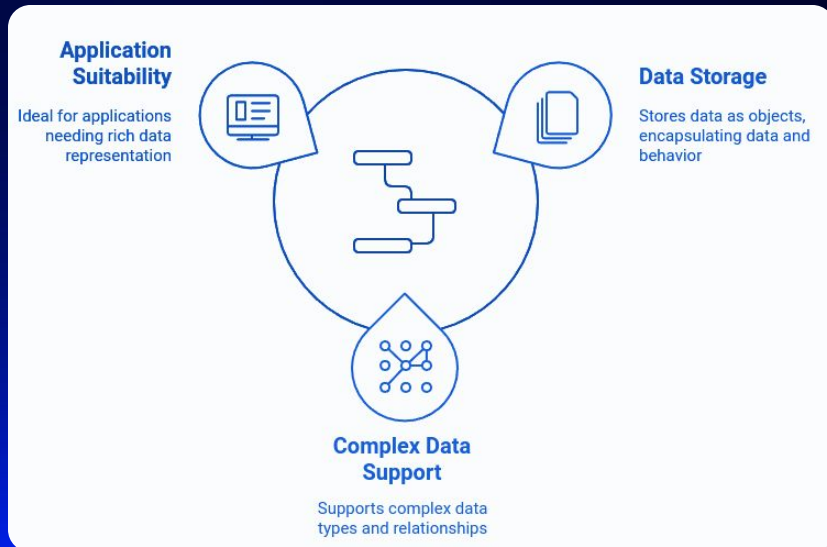
Examples

Integrated Data Store (IDS), TurboIMAGE.



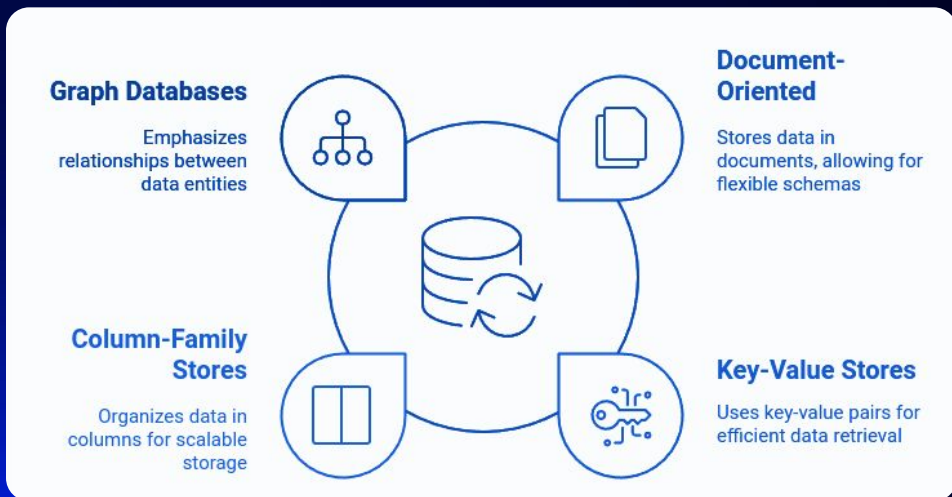
Object-Oriented Databases

- A database model that stores data in the form of objects, as used in object-oriented programming, integrating data and methods while supporting features like inheritance, encapsulation, and polymorphism.



NoSQL Databases

- A non-relational database model designed to handle diverse data types, offering high scalability and flexibility, often used for unstructured, semi-structured, or large-scale distributed data.



Comparison of Database Types

Database Type	Structure	Scalability	Flexibility	Use Cases
Relational (RDBMS)	Tables	Moderate	Limited	Enterprise applications, finance
Hierarchical	Tree-like	Limited	Rigid	Legacy systems
Network	Graph	Moderate	Flexible	Complex relationship modeling
Object-Oriented	Objects	Moderate	High	Applications with complex data types
NoSQL	Varies (documents, key-value)	Highly scalable	Very high	Big Data, real-time web applications

Pop Quiz

Q. Which of these is an example of a NoSQL database?

A

MongoDB

B

MySQL

Pop Quiz

Q. Which of these is an example of a NoSQL database?

A

MongoDB

B

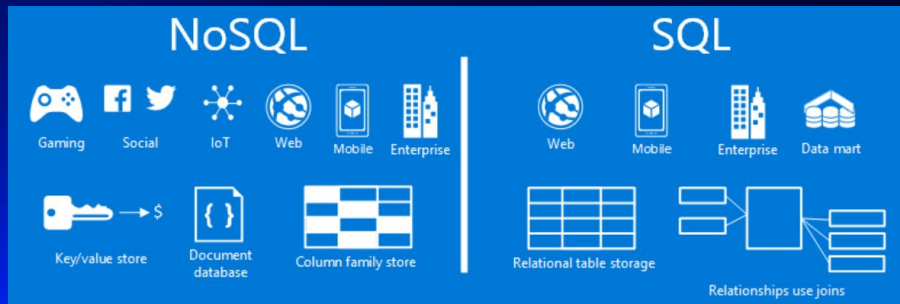
MySQL



Relational vs. Non-Relational Databases

Introduction

- Databases are essential for storing and managing data in various applications.
- They can be broadly classified into two categories:
 - Relational Databases (SQL)
 - Non-Relational Databases (NoSQL)
- Understanding the differences between these types helps organizations choose the right database solution based on their specific needs.



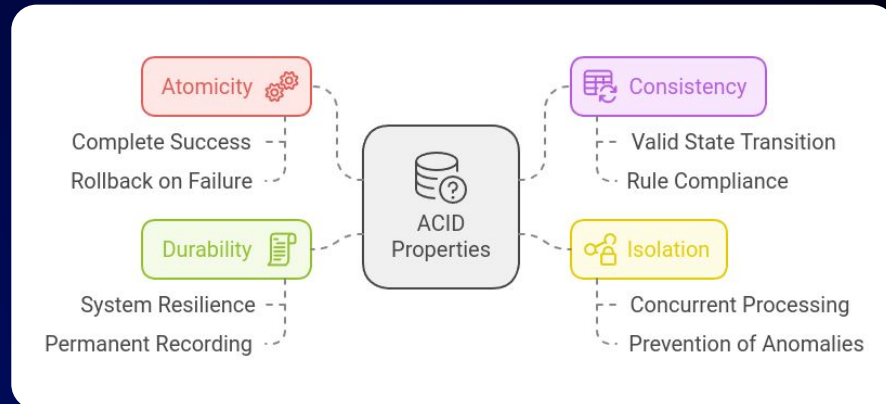
SQL Databases (Relational)

- Relational databases store data in structured tables with rows and columns, using a predefined schema.
- **Key Features:**
 - Use Structured Query Language (SQL) for querying and managing data.
 - Ensure data integrity through ACID properties (Atomicity, Consistency, Isolation, Durability).
 - Suitable for complex queries and multi-row transactions.
- **Examples:** MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server.



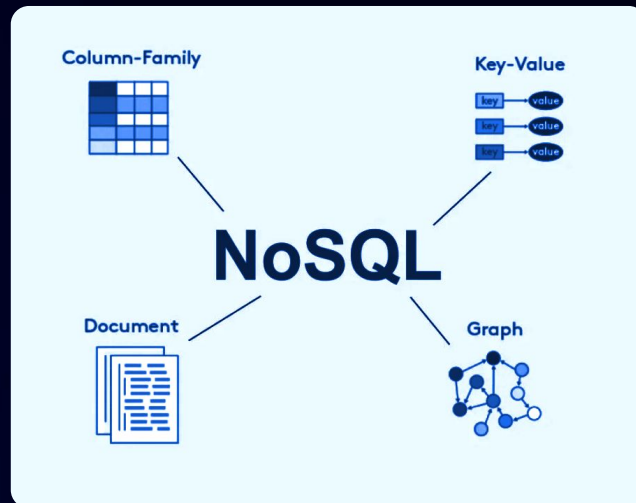
SQL Databases (Relational)

- ACID properties in SQL database
 - ACID compliance is crucial for ensuring the integrity and reliability of transactions in SQL databases.
 - The acronym stands for **Atomicity**, **Consistency**, **Isolation**, and **Durability**, which are the four fundamental properties that guarantee that database transactions are processed reliably.



NoSQL Databases (Non-Relational)

- Non-relational databases offer a flexible approach to data storage, accommodating unstructured or semi-structured data without a fixed schema.
- **Key Features:**
 - Support various data models such as document, key-value, column-family, and graph.
 - Provide horizontal scalability by distributing data across multiple servers.
 - Allow for dynamic schemas, making them ideal for rapidly changing data requirements.
- **Examples:** MongoDB, Cassandra, Redis, Neo4j



Comparison of SQL and NoSQL Databases

Feature	SQL Databases (Relational)	NoSQL Databases (Non-Relational)
Data Model	Tabular	Key-value, document, graph
Schema	Fixed schema	Dynamic schema
Data Type	Structured	Structured, semi-structured, unstructured
Scalability	Vertical scaling	Horizontal scaling
Data Integrity	High with ACID compliance	Eventual consistency
Query Language	SQL	Varies (e.g., MongoDB Query Language)

Best-Suited Scenarios

- **When to Use SQL Databases:**
 - Applications requiring complex queries and transactions involving multiple tables.
 - Scenarios where data integrity and consistency are critical.
 - Systems with well-defined schemas and structured data requirements.
- **When to Use NoSQL Databases:**
 - Applications dealing with large volumes of unstructured or rapidly changing data.
 - Scenarios requiring high scalability and flexibility in data storage.
 - Use cases involving real-time analytics or big data applications.

Pop Quiz

Q. Which type of database is typically used for applications requiring strict schema

A

Non-relational Databases

B

Relational Databases

Pop Quiz

Q. Which type of database is typically used for applications requiring strict schema

A

Non-relational Databases

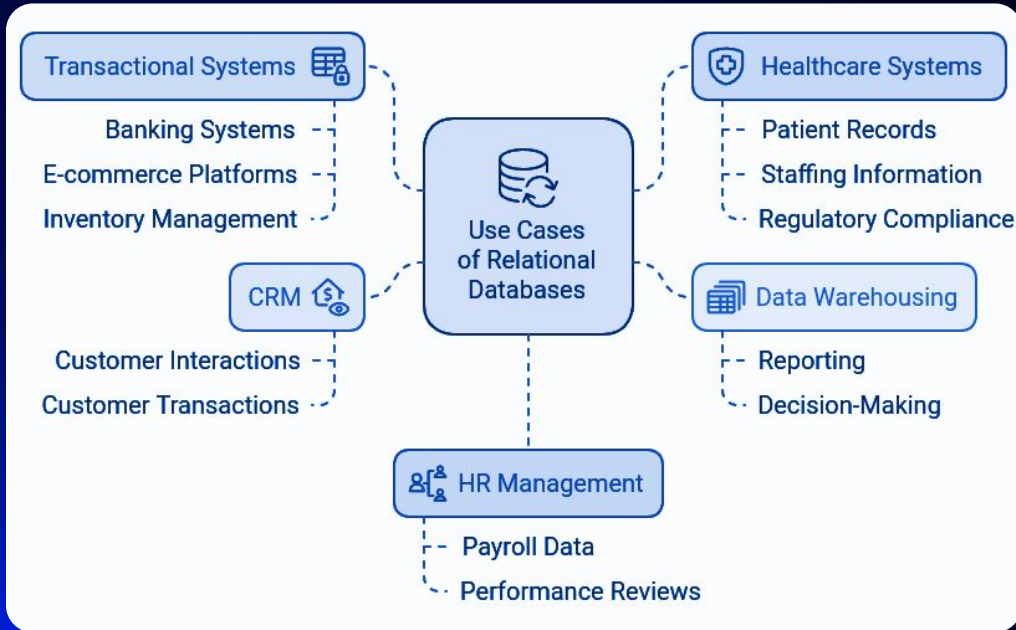
B

Relational Databases

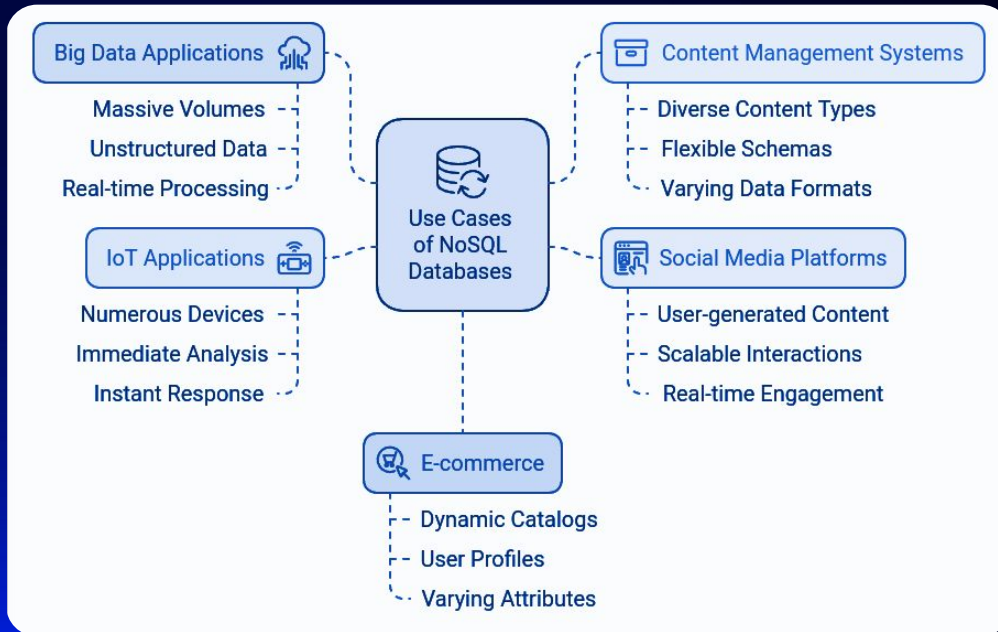


Use Cases of Different Databases

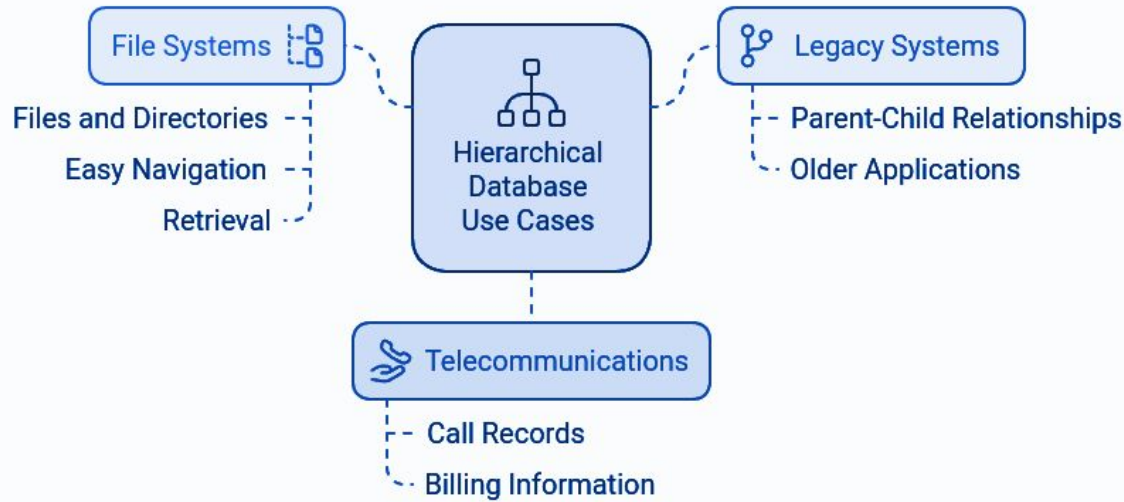
Use Cases of Relational Databases (RDBMS)



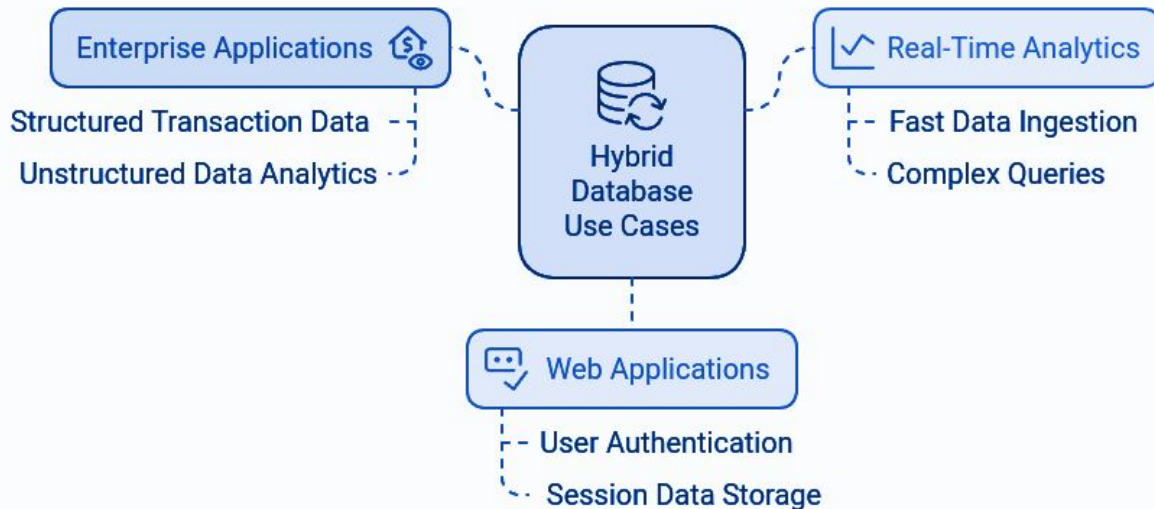
Use Cases of NoSQL Databases



Use Cases of Hierarchical Databases



Hybrid Use Cases



Pop Quiz

Q. Which type of database is best suited for financial applications requiring ACID compliance?

A

Hierarchical Databases

B

Relational Databases

Pop Quiz

Q. Which type of database is best suited for financial applications requiring ACID compliance?

A

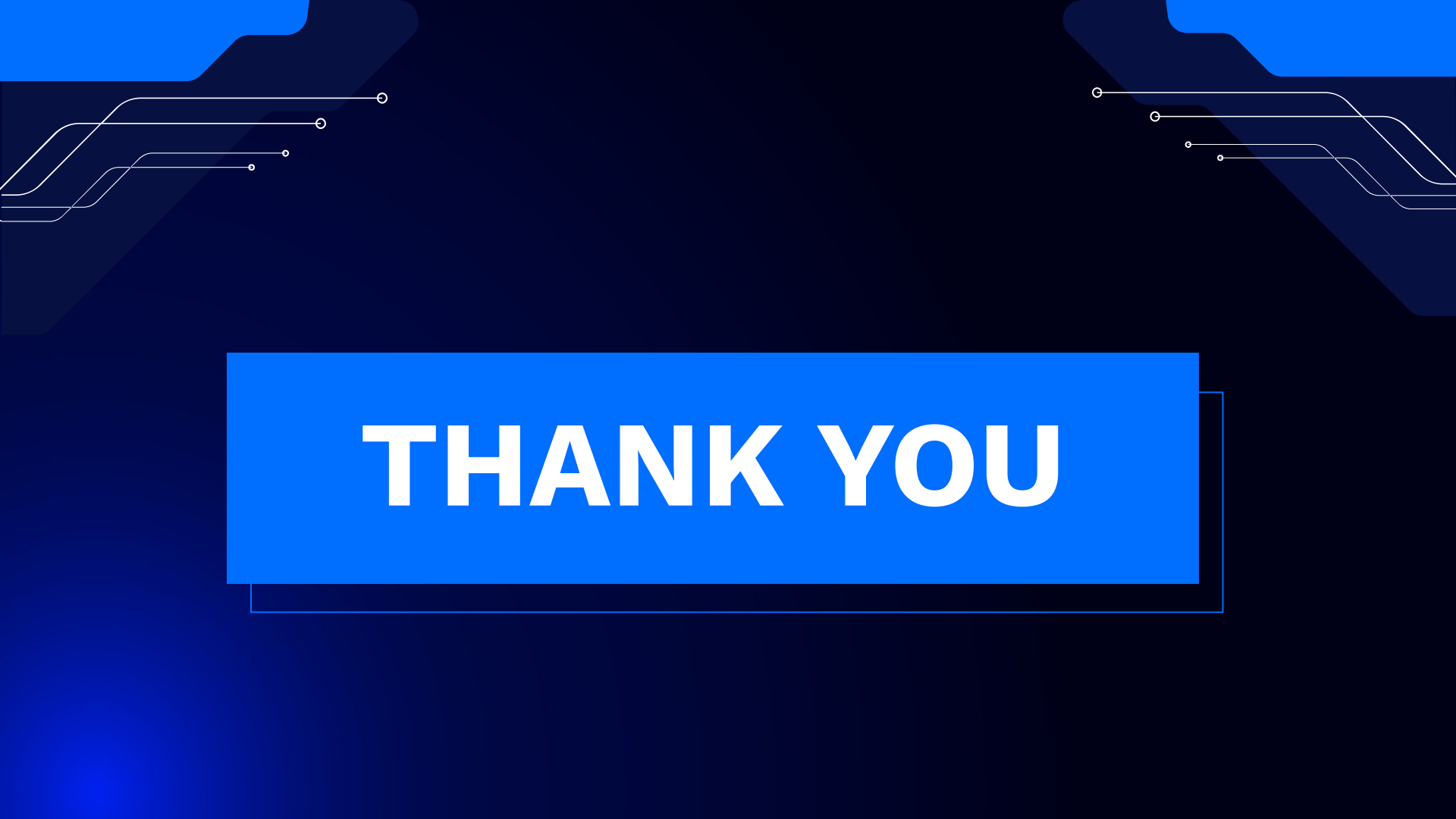
Hierarchical Databases

B

Relational Databases

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



THANK YOU